

CPU Scheduling & Deadlock Avoidance Algorithms

Step-by-step algorithms corresponding to the implemented programs

Banker's Algorithm

- 1 Start.
- 2 Input the number of processes (n) and number of resource types (m).
- 3 Input the Allocation matrix, where $\text{Allocation}[i][j]$ denotes the units of resource j currently held by process i.
- 4 Input the Maximum matrix, where $\text{Maximum}[i][j]$ denotes the maximum units of resource j that process i may need.
- 5 Input the Available array, representing the units of each resource currently free in the system.
- 6 Compute the Need matrix using $\text{Need}[i][j] = \text{Maximum}[i][j] - \text{Allocation}[i][j]$ for all i, j.
- 7 Initialize an empty list of completed (finished) processes.
- 8 Repeat until all processes are completed or no progress can be made in a pass:
 - 9 a. For every process i not yet completed, check if $\text{Need}[i][j] \leq \text{Available}[j]$ for all resources j.
 - 10 b. If such a process is found, assume it can finish: add its Allocation to Available ($\text{Available}[j] += \text{Allocation}[i][j]$ for all j), mark it as completed, and record it in the safe sequence.
 - 11 c. If no process satisfies the condition in a full pass, stop the loop.
- 12 If the number of completed processes equals n, the system is in a Safe State — print "System safe" along with the Safe Sequence.
- 13 Otherwise, the system is in an Unsafe State — print "System in unsafe state".
- 14 Stop.

Code

```
n = int(input("Enter the number of processes:"))
m = int(input("Enter the number of resources:"))

allocation = []
maximum = []
available = []
need = []

print("Enter the allocation matrix: ")
for i in range(n):
    row = []
    for j in range(m):
        value = int(input(f"Enter P{i+1} R{j+1}"))
```

```

        row.append(value)
    allocation.append(row)

print("Enter the maximum matrix: ")
for i in range(n):
    row = []
    for j in range(m):
        value = int(input(f"Enter P{i+1} R{j+1}"))
        row.append(value)
    maximum.append(row)

print("Enter the available resources:")

for j in range(m):
    available.append(int(input(f"R{j+1}: ")))

for i in range(n):
    row = []
    for j in range(m):
        row.append(maximum[i][j] - allocation[i][j])
    need.append(row)

completed = []
while len(completed) < n:
    found = False

    for i in range(n):
        if i not in completed:
            possible = True

            for j in range(m):
                if need[i][j] > available[j]:
                    possible = False

            if possible:
                for j in range(m):
                    available[j] += allocation[i][j]

                completed.append(i)
                found = True

    if found == False:
        break

if len(completed) == n:
    print("\nSystem safe")
    print("Safr Sequence: ", end="")

    for i in completed:
        print(f"P{i+1} ", end=" ")
else:
    print("\nSystem in UnSAFE")

```

Round Robin

- 1 Start.
- 2 Input the number of processes (n).
- 3 For each process, input its Arrival Time (AT) and Burst Time (BT); initialize Remaining Burst Time = BT and Completion Time = 0.

- 4 Input the Time Quantum (TQ).
- 5 Sort the processes by Arrival Time.
- 6 Initialize current_time = 0, a ready queue, and counters for completed processes, total waiting time (TWT), and total turnaround time (TTAT).
- 7 Repeat until all processes are completed:
 - 8 a. Add all processes whose Arrival Time $\leq$ current_time to the ready queue (if not already added).
 - 9 b. If the ready queue is empty, advance current_time to the arrival time of the next unarrived process and continue.
 - 10 c. Remove the process at the front of the queue.
 - 11 d. If its Remaining Burst Time > Time Quantum, execute it for TQ units: increase current_time by TQ and decrease Remaining Burst Time by TQ, then re-insert this process at the back of the queue (after adding any newly arrived processes).
 - 12 e. Otherwise, execute it for its full Remaining Burst Time: increase current_time by Remaining Burst Time, set Remaining Burst Time to 0, and set Completion Time = current_time. The process is now finished.
 - 13 f. For each finished process compute Turnaround Time (TAT) = Completion Time – Arrival Time, and Waiting Time (WT) = TAT – Burst Time; add these to TTAT and TWT and increment the completed counter.
- 14 After all processes complete, compute Average Waiting Time = TWT / n and Average Turnaround Time = TTAT / n.
- 15 Print the process table (AT, BT, CT, TAT, WT) and the two averages.
- 16 Stop.

Code

```
n = int(input("Enter the number of processes: "))
processes = []

for i in range(n):
    at = int(input("Enter the arrival time(AT) for P{i+1}: "))
    bt = int(input("Enter the burst time(BT) for P{i+1}: "))
    processes.append([i+1, at, bt, bt, 0])

time_quantum = int(input("\nEnter the time quantum: "))

processes.sort(key=lambda x:x[1])

current_time = 0
completed = 0
tw = 0
ttat = 0

queue = []
index = 0

print("\nProcess\tAT\tBT\tCT\tTAT\tWT")
```

```

while completed < n:
    while index < n and processes[index][1] <= current_time:
        queue.append(processes[index])
        index += 1

    if len(queue) == 0:
        current_time = processes[index][1]
        continue

    process = queue.pop(0)

    pid, at, bt, remaining_bt, ct = process
    if remaining_bt > time_quantum:
        current_time += time_quantum
        process[3] -= time_quantum

    else:
        current_time += remaining_bt
        process[3] = 0

        process[4] = current_time

        ct = process[4]
        tat = ct - at
        wt = tat - bt

        twt += wt
        ttat += tat

        completed += 1

        print(f"P{pid}\t{at}\t{bt}\t{ct}\t{tat}\t{wt}")

    while index < n and process[index][1] <= current_time:
        queue.append(processes[index])
        index += 1

    if process[3] > 0:
        queue.append(process)

avg_wt = twt / n
avg_tat = ttat / n

print(f"\nAverage waiting time = {avg_wt:.2f}")
print(f"Average turnaround time = {avg_tat:.2f}")

```

SJF Preemptive

- 1 Start.
- 2 Input the number of processes (n).
- 3 For each process, input its Arrival Time (AT) and Burst Time (BT); initialize Remaining Burst Time = BT and Completion Time = 0.
- 4 Sort the processes by Arrival Time.
- 5 Initialize current_time = 0, completed = 0, and counters for total waiting time (TWT) and total turnaround time (TTAT).

- 6 Repeat until all processes are completed:
- 7 a. Build a list of "available" processes — those with Arrival Time $\leq$ current_time and Remaining Burst Time > 0 .
- 8 b. If no process is available, increment current_time by 1 and continue (CPU idle).
- 9 c. Otherwise, select the available process with the smallest Remaining Burst Time.
- 10 d. Execute the selected process for one time unit: decrease its Remaining Burst Time by 1 and increase current_time by 1.
- 11 e. If its Remaining Burst Time becomes 0, set Completion Time = current_time, compute Turnaround Time (TAT) = Completion Time – Arrival Time and Waiting Time (WT) = TAT – Burst Time, add these to TTAT and TWT, and increment completed.
- 12 After all processes complete, compute Average Waiting Time = TWT / n and Average Turnaround Time = TTAT / n.
- 13 Print the process table (AT, BT, CT, TAT, WT) and the two averages.
- 14 Stop.

Code

```
n = int(input("Enter the number of processes: "))
processes = []

for i in range(n):
    at = int(input(f"Enter Arrival time (AT) for P{i+1}: "))
    bt = int(input(f"Enter Burst time (BT) for P{i+1}: "))
    processes.append([i+1, at, bt, bt, 0])

processes.sort(key=lambda x: x[1])

current_time = 0
completed = 0
twt = 0
ttat = 0

print(f"\nProcess\tAT\tBT\tCT\tTAT\tWT")

while completed < n:

    available = []

    for process in processes:
        if process[1] <= current_time and process[3] > 0:
            available.append(process)

    if len(available) == 0:
        current_time += 1
        continue

    available.sort(key=lambda x: x[3])

    process = available[0]

    pid, at, bt, remaining_bt, ct = process

    process[3] -= 1
```

```

current_time += 1

if process[3] == 0:

    process[4] = current_time

    ct = process[4]
    tat = ct - at
    wt = tat - bt

    twt += wt
    ttat += tat

    completed += 1

    print(f"P{pid}\t{at}\t{bt}\t{ct}\t{tat}\t{wt}")

avg_wt = twt / n
avg_tat = ttat / n

print(f"\nAverage waiting time = {avg_wt:.2f}")
print(f"Average turnaround time = {avg_tat:.2f}")

```

SJF Non-Preemptive

- 1 Start.
- 2 Input the number of processes (n).
- 3 For each process, input its Arrival Time (AT) and Burst Time (BT).
- 4 Sort the processes by Arrival Time.
- 5 Initialize current_time = 0, and counters for total waiting time (TWT) and total turnaround time (TTAT).
- 6 Repeat until every process has been scheduled:
 - 7 a. If the CPU is idle (current_time is less than the arrival time of the earliest remaining process), advance current_time to that arrival time.
 - 8 b. Build a list of "available" processes — those with Arrival Time $\leq$ current_time.
 - 9 c. Among the available processes, select the one with the smallest Burst Time.
 - 10 d. Execute the selected process to completion (no preemption): Completion Time (CT) = current_time + Burst Time.
 - 11 e. Compute Turnaround Time (TAT) = CT – AT and Waiting Time (WT) = TAT – BT; add these to TTAT and TWT.
 - 12 f. Update current_time = CT and remove the selected process from the pending list.
- 13 After all processes complete, compute Average Waiting Time = TWT / n and Average Turnaround Time = TTAT / n.
- 14 Print the process table (AT, BT, CT, TAT, WT) and the two averages.

15 Stop.

Code

```
n = int(input("Enter the number of processes: "))
processes = []

for i in range(n):
    at = int(input(f"Enter Arrival time (AT) for P{i+1}: "))
    bt = int(input(f"Enter Burst time (BT) for P{i+1}: "))
    processes.append([i+1, at, bt])

# First sort by Arrival Time
processes.sort(key=lambda x: x[1])

current_time = 0
twt = 0
ttat = 0

print("\nProcess\tAT\tBT\tCT\tTAT\tWT")

while len(processes) > 0:

    # If CPU is idle, take the first arrived process
    if current_time < processes[0][1]:
        current_time = processes[0][1]

    # Find all processes that have arrived
    available = []

    for process in processes:
        if process[1] <= current_time:
            available.append(process)

    # Sort available processes by Burst Time
    available.sort(key=lambda x: x[2])

    # Take the process with shortest BT
    process = available[0]

    pid, at, bt = process

    ct = current_time + bt
    tat = ct - at
    wt = tat - bt

    twt += wt
    ttat += tat

    print(f"P{pid}\t{at}\t{bt}\t{ct}\t{tat}\t{wt}")

    current_time = ct

    # Remove completed process
    processes.remove(process)

avg_wt = twt / n
avg_tat = ttat / n

print(f"\nAverage waiting time = {avg_wt:.2f}")
print(f"Average turnaround time = {avg_tat:.2f}")
```

FCFS

- 1 Start.
- 2 Input the number of processes (n).
- 3 For each process, input its Arrival Time (AT) and Burst Time (BT).
- 4 Sort the processes in ascending order of Arrival Time.
- 5 Initialize current_time = 0, and counters for total waiting time (TWT) and total turnaround time (TTAT).
- 6 For each process, taken in arrival order:
 - 7 a. If current_time < Arrival Time (AT) of the process, advance current_time to AT (CPU idle until process arrives).
 - 8 b. Compute Completion Time (CT) = current_time + Burst Time (BT).
 - 9 c. Compute Turnaround Time (TAT) = CT – AT and Waiting Time (WT) = TAT – BT.
 - 10 d. Add TAT and WT to TTAT and TWT respectively.
 - 11 e. Update current_time = CT and move to the next process.
- 12 After all processes are scheduled, compute Average Waiting Time = TWT / n and Average Turnaround Time = TTAT / n.
- 13 Print the process table (AT, BT, CT, TAT, WT) and the two averages.
- 14 Stop.

Code

```
n = int(input("Enter the number of processes"))
processes = []

for i in range(n):
    at = int(input(f"Enter Arrival time (AT) for P{i+1}: "))
    bt = int(input(f"Enter the busrt time (BT) for P{i+1}: "))
    processes.append([i+1,at,bt])

processes.sort(key=lambda x:x[1])
current_time = 0
twt = 0
ttat = 0

print(f"\nProcess\tAT\tBT\tCT\tTAT\tWT")

for process in processes:
    pid, at, bt = process

    if current_time < at:
        current_time = at

    ct = current_time + bt
    tat = ct - at
    wt = tat - bt
    twt += wt
    ttat += tat
```



```
        print(f"P{pid}\t{at}\t{bt}\t{ct}\t{tat}\t{wt}")
        current_time = ct

avg_wt = twt / n
avg_tat = ttat / n

print(f"\nAverage waiting time = {avg_wt:.2f}")
print(f"Average turnaround time = {avg_tat:.2f}")
```